EPSI • RNCP35584 NIVEAU 7 • BLOC 4 — MSPR TPRE841 (LIVRABLE 06 — COMPETENCES C5, C6 & C7)

Plan de Tests, Validation Globale et Conformité — MECHA Predict (C5 & C7)

Projet : MECHA — Maintenance Prédictive Industrielle par IA • **Référence** : MSPR 2 — Bloc 4 RNCP35584 (`TPRE841`) | **Version** : 2.0 • **Compétences couvertes** : **C5 — Effectuer les tests et établir les plans de correction** & **C7 — Vérifier la conformité au cahier des charges**

1. Stratégie et typologie des tests (Critère C5)

Conformément au contexte industriel critique de MECHA (aéronautique EN 9100 et automobile IATF 16949), la validation repose sur une **pyramide de tests automatisée à 4 niveaux** exécutée à chaque commit via `pytest` et GitHub Actions :

Typologie de test	Fichier de test réel	Nb tests	Objectif de vérification	Seuil d'acceptation
1. Qualité & intégrité des données (Unitaire Data)	`tests/test_data_quality.py`	**26**	Schéma 30 colonnes, 110 000 lignes, bornes physiques capteurs, 5 usines FR/ES, absence de NaN sur les 10 features ML, compatibilité stricte PostgreSQL `\COPY` (`INTEGER`)	**100 % PASS**
2. Modèles ML & anti-régression (Unitaire ML)	`tests/test_model.py`	**17**	Chargement des `.joblib` , dimensions d'entrée (10 features), sortie binaire `[0, 1]` , probabilités `[0, 1]` , alignement `MODEL_FILES` ↔ `MODEL_FILENAMES` , absence de mode mock silencieux	**100 % PASS**
3. API REST & contrat Pydantic (Unitaire / Composant)	`api/tests/test_api.py`	**55**	Validation des 9 endpoints (`/health` , `/predict` , `/predict/batch` , `/predict/rul` , `/anomaly` , `/metrics` , `/model-info` , `GET/PUT /alerts/config`), rejet HTTP `422` hors bornes, règles métier	**100 % PASS** & Couverture API >= 80 %
4. Chaîne complète & Recette Métier (Intégration E2E)	`tests/test_integration.py` + Recette UI	**6** (+ 8 scénarios UAT)	Cohérence bout-en-bout machine normale vs machine critique, monotonie du RUL (`RUL_normal > RUL_critique`), cohérence unitaire vs batch, recette fonctionnelle Streamlit & Grafana	**100 % PASS**
TOTAL AUTOMATISÉ	Suite `pytest` complète	**104**	Vérification systématique avant construction des images Docker	**104 / 104 PASS**

2. Outillage de testing retenu (Critère C5)

Outil	Version	Rôle dans le dispositif de validation
pytest	`>= 7.4` (`9.2`)	Framework principal d'exécution des tests unitaires, composants et intégration (fixtures modulaires, paramétrage)
pytest-cov / coverage	`>= 4.1` (`7.1`)	Mesure de couverture d'instructions (`--cov=api --cov=models`) avec génération d'artefacts XML et HTML en CI
FastAPI TestClient (`httpx`)	`>= 0.25`	Requêtage HTTP synchrone/asynchrone in-process de l'application ASGI FastAPI sans latence réseau
Ruff	`>= 0.1` (`0.17`)	Analyse statique (`ruff check` règles `E4, E7, E9, F, I`) et contrôle de formatage (`ruff format --check`)
GitHub Actions	v4	Orchestration CI automatique sur `ubuntu-latest` (Python 3.11) bloquant le build Docker en cas d'échec

3. Jeux d'essai de référence (Données de test et résultats attendus)

3.1 Jeux d'essai synthétiques déterministes (API & Intégration)

Définis dans `tests/test_integration.py` et `api/tests/test_api.py` aux échelles exactes du dataset MECHA :

Jeu d'essai	Valeurs capteurs injectées	Résultat attendu (`/predict` , `/predict/rul` , `/anomaly`)	Résultat obtenu
`SAMPLE_NORMAL` (Machine nominale)	`temperature=75.0 °C` , `vibration=35.0 mm/s` , `humidity=55.0 %` , `pressure=3.0 bar` , `energy_consumption=2.5 kWh` , `temp_rolling_10min=74.0` , `temp_trend_1h=0.01` , `vibr_rolling_10min=34.0` , `temp_std_30min=1.5` , `energy_vibr_ratio=0.07`	HTTP `200` , `prediction="normal"` , aucune alerte critique, `rul_minutes` élevé (`> 300 min` , catégorie `safe`)	**Conforme**
`SAMPLE_CRITICAL` (Machine en dérive sévère)	`temperature=115.0 °C` , `vibration=85.0 mm/s` , `humidity=60.0 %` , `pressure=1.0 bar` , `energy_consumption=6.5 kWh` , `temp_rolling_10min=112.0` , `temp_trend_1h=0.8` , `vibr_rolling_10min=82.0` , `temp_std_30min=8.0` , `energy_vibr_ratio=0.08`	HTTP `200` , déclenchement d'alertes critiques (`temperature > 100 °C`), `is_anomaly=True` ou sévérité `high/critical` , `rul_minutes < RUL(SAMPLE_NORMAL)`	**Conforme**
`PAYLOAD_INVALID_TEMP` (Sonde hors bornes)	`temperature=350.0 °C` (hors plage Pydantic `[-50, 250]`) ou champ requis manquant	Rejet immédiat HTTP `422 Unprocessable Entity` avant passage au modèle ML	**Conforme**
`BATCH_OVER_LIMIT` (Surcharge lot)	Lot de `101` machines sur `POST /predict/batch` (limite fixée à `100`)	Rejet contrôlé HTTP `422` protégeant la mémoire du conteneur API	**Conforme**
`ALERT_CONFIG_UPDATE` (Paramétrage C4)	`PUT /alerts/config` avec `{"temperature_critical_celsius": 95.0, "trs_min_percent": 80.0}`	HTTP `200` , mise à jour immédiate en mémoire et persistance vérifiée via `GET /alerts/config`	**Conforme**

3.2 Jeu d'essai d'évaluation ML (Split temporel strict sans fuite)

- **Corpus total** : `110 000` lignes (`100 000` simulées MECHA + `10 000` du benchmark public UCI AI4I 2020).

- **Protocole de découpage** : Split chronologique **80 % entraînement (`88 000` lignes) / 20 % test (`22 000` lignes)** appliqué **séparément par source** (`20 000` lignes simulées + `2 000` lignes AI4I en test, soit `1 039` événements positifs de maintenance).
- **Exclusion anti-fuite vérifiée** : `machine_status` , `predicted_remaining_life` et `downtime_risk` sont strictement exclus des **10 features capteurs**.

Modèle évalué	Métrique cible cahier des charges	Résultat obtenu sur Test (n = 22 000)	Statut
Random Forest Classifier (modèle par défaut)	`F1 >= 0,75` , `Recall >= 0,85`	**F1 = 80,43 %**, **Recall = 89,99 %**, Précision = 72,71 %, AUC = 98,80 %	**Validé**
XGBoost Classifier (modèle alternatif)	`F1 >= 0,75` , `Recall >= 0,85`	**F1 = 81,11 %**, **Recall = 88,64 %**, Précision = 74,76 %, AUC = 98,83 %	**Validé**
Régression Logistique (baseline linéaire)	Référence comparative	F1 = 23,33 %, Recall = 92,01 %, Précision = 13,36 % (justifie le recours aux forêts/boosting)	**Validé**
Isolation Forest (anomalies non supervisé)	Détection d'atypicité ~5 %	Contamination effective = **5,57 %** (`1 226` alertes), AUC = **83,86 %**	**Validé**
Random Forest Regressor (RUL)	`MAE <= 45 min` , `R² >= 0,65`	**MAE = 26,94 min**, RMSE = 46,36 min, **R² = 0,7189** (échelle 0–500 min)	**Validé**

4. Extraits représentatifs des tests réels implémentés

4.1 Test de non-régression sur le branchement réel des modèles (`tests/test_model.py`)

```
def test_training_and_api_filenames_match(self):
    from api.main import MODEL_FILES
    # Verifie que chaque modele sauvegarde par train_models.py porte exactement
    # le nom attendu par l'API FastAPI (empeche tout repli silencieux en mode mock)
    for key, api_filename in MODEL_FILES.items():
        assert key in train.MODEL_FILENAMES
        expected = f"{train.MODEL_FILENAMES[key]}.joblib"
        assert api_filename == expected
```

4.2 Test de compatibilité d'import SQL PostgreSQL `\COPY` (`tests/test_data_quality.py`)

```
def test_integer_columns_have_no_float_suffix_in_csv(self):
    raw_ints = pd.read_csv(
        csv_path,
        usecols=["rotational_speed_rpm", "tool_wear_min", "machine_id"],
        dtype=str,
        keep_default_na=False,
    )
    for col in ["rotational_speed_rpm", "tool_wear_min", "machine_id"]:
        non_empty = raw_ints[col][raw_ints[col] != ""]
        bad = non_empty[~non_empty.str.fullmatch(r"-?\d+")]
        assert len(bad) == 0
```

4.3 Test d'intégration bout-en-bout (`tests/test_integration.py`)

```
def test_rul_prediction_coherent(self, client):
    resp_normal = client.post("/predict/rul", json=SAMPLE_NORMAL)
    resp_critical = client.post("/predict/rul", json=SAMPLE_CRITICAL)
    assert resp_normal.json()["rul_minutes"] > resp_critical.json()["rul_minutes"]
```

5. Scénarios de tests de recette utilisateur (UAT — Métiers MECHA)

ID	Scénario métier testé	Profil testeur	Étapes exécutées sur le prototype	Résultat
UAT-01	Supervision globale multi-sites	Direction Industrielle	Ouverture vue Streamlit **Groupe** + dashboard Grafana **DG** : affichage des 5 usines FR/ES, 50 machines simulées + parc benchmark, KPIs temps réel	**PASS**
UAT-02	Pilotage d'atelier local	Responsable Site Lyon	Sélection `Lyon (FR)` dans la vue **Site** : filtrage instantané des machines du site, lecture du statut et des probabilités IA	**PASS**
UAT-03	Diagnostic prédictif d'une machine fragile	Technicien Maintenance	Sélection d'une machine `vieillissante` (ex. Machine #5) dans la vue **Machine** : appel temps réel `/predict` , `/predict/rul` (en min) et `/anomaly` + jauges capteurs	**PASS**
UAT-04	Filtrage opérationnel des alertes	Chef d'équipe	Vue **Alertes** : filtrage combiné par usine + type (`Prediction IA` , `Temperature critique` , `RUL critique` , `Panne`)	**PASS**
UAT-05	Validation *Human-in-the-Loop* & retour terrain (C8)	Technicien / Opérateur	Vue **Accompagnement → Qualification Terrain** : enregistrement d'un verdict (*Vrai positif* / *Fausse alerte*) sans donnée personnelle et mise à jour du journal	**PASS**
UAT-06	Paramétrage dynamique des seuils métier (C4)	Responsable Maintenance	Vue **Accompagnement → Paramétrage Seuils API** : modification du seuil température critique via `PUT /alerts/config` et prise en compte immédiate	**PASS**
UAT-07	Transparence et auditabilité IA	Data Scientist / DPO	Vue **Modèle IA** + appel `GET /model-info` : affichage des 4 modèles chargés, métriques par source et mention explicite *Human-in-the-Loop*	**PASS**
UAT-08	Continuité en mode dégradé	DSI / Exploitation	Arrêt volontaire du conteneur API : bandeau latéral signalant explicitement `API IA Indisponible` , maintien de la consultation des mesures capteurs sans crash	**PASS**

6. Plan de correction et d'amélioration avant mise en production (Critère C5)

Les campagnes de tests unitaires, d'intégration et de revue pré-production ont permis d'identifier et de corriger **5 dysfonctionnements majeurs** avant livraison :

Réf.	Dysfonctionnement identifié lors des tests	Sévérité	Action corrective appliquée	Test de non-régression associé	Statut
CORR-01	**Fuite de données (*Data Leakage*)** : inclusion initiale de `predicted_remaining_life` et `downtime_risk` dans les features ML gonflant artificiellement le F1 à >99 %	Critique	Retrait strict des 3 colonnes de fuite (`LEAKY_FEATURES`), passage à **10 features capteurs pures** et split temporel 80/20 par source	`test_no_machine_status_leakage_in_features` , vérification `n_features_in_ == 10`	**Corrigé**
CORR-02	**Repli silencieux en mode Mock de l'API** : écart de nommage entre `rf_classifier.joblib` (script train) et `random_forest_classifier.joblib` (API)	Critique	Alignement strict `MODEL_FILENAMES` ↔ `MODEL_FILES` + étape CI bloquante vérifiant `health["models_loaded"] == True`	`TestModelFilenamesConsistency` (`tests/test_model.py`)	**Corrigé**
CORR-03	**Échec d'import PostgreSQL `\COPY` (`2636.0`)** : les `NaN` des lignes simulées forçaient pandas à exporter `rotational_speed_rpm` et `tool_wear_min` en flottants, annulant l'import SQL pour Grafana	Critique	Conversion explicite en entiers nullables pandas `Int64` et ordre déterministe des 30 colonnes dans `merge_datasets.py`	`TestPostgreSQLCopyCompatibility` (`tests/test_data_quality.py`)	**Corrigé**
CORR-04	**Échec `docker compose up` sur clone vierge** : le montage direct d'un fichier CSV absent créait un dossier vide bloquant l'initialisation	Majeure	Montage du dossier `./data/processed:/data/processed:ro` et ajout du service d'amorçage automatique `init-pipeline` dans `docker-compose.yml`	Validation `docker compose config` & démarrage à froid	**Corrigé**
CORR-05	**Incohérence d'unité RUL (heures vs minutes)** entre le dataset (0–500 min) et l'API	Majeure	Standardisation complète sur `rul_minutes` (seuils métier `50 / 150 / 300 min`) dans l'API, Streamlit, PostgreSQL et la documentation	`test_rul_prediction_coherent` & `test_api.py`	**Corrigé**

Plan d'amélioration continue post-pilote (Industrialisation M3 → M12)

1. **Enrichissement capteurs usinage** : intégrer directement le couple broche (`torque_nm`) et l'usure outil (`tool_wear_min`) sur les automates des 5 usines pour aligner les performances réelles sur le benchmark AI4I (>85 % F1).
2. **Détection automatique de dérive (*Data / Concept Drift*)** : planifier un job hebdomadaire calculant le *Population Stability Index (PSI)* sur les 10 features capteurs et déclenchant un ré-entraînement si baisse de F1 > 5 %.
3. **Connecteur bidirectionnel GMAO** : pré-remplir automatiquement les bons de travail dans la GMAO après validation humaine dans l'onglet *Qualification Terrain*.

---

7. Matrice de conformité Réalisation vs Cahier des Charges (Critère C7)

Exigence du Cahier des Charges (`TPRE841`)	Réalisation technique dans MECHA Predict	Preuve / Livrable associé	Conformité
Collecte & préparation structurée de données publiques/simulées (aucune donnée réelle confidentielle)	Corpus hybride de 110 000 lignes × 30 colonnes combinant simulation Python (`seed=42`) et benchmark public UCI AI4I 2020	`generate_data.py` , `merge_datasets.py` , `data_dictionary.md`	**100 % Conforme**
Architecture intégrant capteurs IoT, SCADA/MES (cohérence temporelle) et stockage centralisé	Collecte MQTT/OPC-UA synchronisée NTP via SCADA/MES, stockage centralisé PostgreSQL 15 indexé + vues SQL	`architecture.md` , `db/init.sql` , `docker-compose.yml`	**100 % Conforme**
Classification de l'état machine (normal / à risque)	Random Forest (`F1 = 80,4 %`) et XGBoost (`F1 = 81,1 %`) sur 10 features capteurs	`train_models.py` , `POST /predict` , `POST /predict/batch`	**100 % Conforme**
Prédiction d'un indicateur de dégradation (temps restant avant défaillance)	Random Forest Regressor prédisant le RUL en minutes (`MAE = 26,9 min` , `R² = 0,72`)	`rf_regressor_rul_card.md` , `POST /predict/rul`	**100 % Conforme**
Modèle de comparaison simple + détection de comportements atypiques	Régression Logistique baseline (`F1 = 23,3 %`) + Isolation Forest non supervisé (`AUC = 83,9 %`)	`logistic_regression_card.md` , `isolation_forest_card.md` , `POST /anomaly`	**100 % Conforme**
Solution applicative intégrée, paramétrage applicatif et formalisme de configuration (C4)	API FastAPI + Streamlit (6 vues) + Grafana (3 dashboards) provisionnés en YAML/JSON/SQL + seuils configurables via `GET/PUT /alerts/config`	`api/main.py` , `app/src/dashboard.py` , `grafana/provisioning/`	**100 % Conforme**
Conteneurisation Docker & chaîne CI minimale (tests + build image)	`docker-compose.yml` (5 services dont init auto) + workflow GitHub Actions 3 jobs (Ruff, Pytest 104 tests, Buildx Docker)	`docker-compose.yml` , `.github/workflows/ci.yml`	**100 % Conforme**
Documentation métier, RGPD/DPO, déploiement multi-sites et conduite du changement outillée (C7/C8)	Guide métier 2 pages, analyse RGPD/AI Act, plan 4 phases FR/ES, module C8 intégré dans Streamlit	`user_guide.md` , `rgpd_analysis.md` , `deployment_plan.md` , `change_management.md`	**100 % Conforme**

8. Rapport d'exécution final de validation

Champ	Valeur
Date d'exécution	Mai 2026 (dernière passe de vérification complète avant remise)
Environnement	Linux Debian 12 — Python `3.11.2`, `pytest 9.2.0`, `ruff 0.17.4`, Docker Compose v2
Analyse statique Ruff	`ruff check .` : **0 erreur** | `ruff format --check .` : **100 % conforme**
Tests exécutés	**104 tests exécutés** (`55` API + `26` Data Quality + `17` Modèles ML + `6` Intégration)
Tests réussis / échoués / ignorés	**104 PASS (100 %)** | **0 FAIL** | **0 SKIP** (durée totale : `8,79 s`)
Couverture de code mesurée	**`api/main.py` : 87 %** (`457` instructions, `398` couvertes) | **`api/tests/test_api.py` : 100 %**
Recette fonctionnelle (UAT)	**8 / 8 scénarios métiers validés** (Streamlit 6 vues + Grafana 3 dashboards + mode dégradé)
Décision de mise en production	**[X] GO DÉPLOIEMENT PILOTE (Phase 1 — Usine MECHA Lyon, 10 machines)**